Homogeneous variadic function parameters

Document #:	P1219R3 [Latest] [Status]
Date:	2026-10-21
Project:	Programming Language C++
Audience:	EWG

Authors:	Ian Pike
	<ianpike98cs@gmail.com>
	Eczbek
	<eczbek.void@gmail.com>
	Jan Schultke
	<janschultke@gmail.com>
Original author:	James Touton
	<bekenn@gmail.com>

Contents

1 Revision history **2**
- 1.1 R3 (2026-10-21) . . . 2
- 1.2 R2 (2019-10-05) . . . 3
- 1.3 R1 (2019-03-11) . . . 3
- 1.4 R0 (2018-10-08) . . . 3

2 Introduction **3**

3 Library use cases **4**
- 3.1 Combine call-site diagnostics with count-dependent results . . . 4
- 3.2 Preserve conversion ranking without fixed-arity wrappers . . . 6
- 3.3 Broader demand for fixed-type packs . . . 7

4 Sequence alternatives **7**

5 Design decisions **9**
- 5.1 Definitions . . . 9
- 5.2 Pack declarations . . . 10
  - 5.2.1 Alternatives . . . 11
  - 5.2.2 Type deduction . . . 12
  - 5.2.3 Target types for initialization . . . 13
  - 5.2.4 Address-taking . . . 13
  - 5.2.5 Overload ordering . . . 14
  - 5.2.6 Constructors . . . 15
- 5.3 Lambdas . . . 16
- 5.4 Pack indexing . . . 17
  - 5.4.1 Array parameters . . . 18

6 Reclaiming the deprecated comma-less spelling **18**
6.1 Current rules . . . 18
6.2 Proposed change . . . 19
6.2.1 Migrating definitions and function types . . . 19

7 Straw polls **20**
7.1 Proposed EWG polls . . . 20
7.2 Historical results . . . 20

8 Implementation experience **21**

9 Wording **21**
9.1 Lambdas . . . 21
9.1.1 General [expr.prim.lambda.general] . . . 21
9.1.2 Closure types [expr.prim.lambda.closure] . . . 22
9.2 Functions [dcl.fct] . . . 22
9.3 Removal and reclamation . . . 23
9.4 Address of an overload set [over.over] . . . 24
9.5 Variadic templates [temp.variadic] . . . 24
9.6 Dependency . . . 25
9.7 Deduction . . . 25
9.7.1 General [temp.deduct.general] . . . 25
9.7.2 Deducing template arguments from a function call [temp.deduct.call] . . . 26
9.7.3 Deducing template arguments from a type [temp.deduct.type] . . . 27
9.7.4 Partial ordering of function templates [temp.func.order] . . . 27
9.7.5 Deducing template arguments during partial ordering [temp.deduct.partial] . . . 27
9.8 Declarations . . . 27
9.9 Feature-test macro [cpp.predefined] . . . 28
9.10 Annex C (informative) Compatibility [diff.cpp26] . . . 28

10 Conclusion **28**

11 Acknowledgments **28**

12 References **28**

Abstract

We propose homogeneous function parameter packs for C++29: a variable number of parameters of one declared type. One declaration replaces fixed-arity overload families, retaining ordinary parameter initialization, conversion ranking, and a compile-time count.

We ask the Evolution Working Group to first confirm the C++29 pairing of named and unnamed packs with mandatory C-varargs commas and reclamation of the comma-less spelling. This gives packs the same meaning with or without a name, while changing declarations deprecated in C++26.

1 Revision history

1.1 R3 (2026-10-21)

— Updated active contributors and credited James Touton as original author.
— Retargeted to C++29 and rebased wording on the current working draft.

— Replaced the separate pack-length component of specialization identity with an invented constant template parameter for the count.
— Added Abseil and CasADi comparisons with current C++ alternatives.
— Updated classification, deduction, initialization, address-taking, and lambda rules for the count model and C++26 pack indexing.
— Required mandatory C-varargs commas and reclamation of the comma-less spelling for unnamed packs in C++29.

1.2 R2 (2019-10-05)

— Rebased wording against N4830.
— Discussed the impact of mandatory varargs commas.

1.3 R1 (2019-03-11)

— Rebased wording against N4800 and added signature changes.
— Removed permission for empty template parameter lists in template declarations.
— Added a feature test macro and alternatives addressing confusion with explicit specializations.
— Added Kona Evolution Working Group Incubator (EWGI) polls and acknowledgments.
— Corrected examples.

1.4 R0 (2018-10-08)

— Initial draft. [P1219R0]

2 Introduction

Homogeneous function parameter packs would replace fixed-arity overload families with one declaration for C++29. Each pack has one declared element type with ordinary parameter initialization and conversions.

Table 1: Independent parameter-pack declarations. C++26 permits all but the proposed lower-right form.

Element types	Template parameter pack	Function parameter pack
Heterogeneous	`template <auto... vs>` `void f();`	`void f(auto... vs);`

Homogeneous	`// Legal since C++11: T is not a parameter pack, but vs is.`
	`template <class T, T... vs>`
	`void foo();`
		`// Proposed (ill-formed in C++26): vs is a pack of T parameters.`
		`template <class T>`
		`void foo(T... vs);`

Table 1's lower-right declaration is ill-formed because `T` names one type, leaving `T... vs` no pack to expand. [temp.variadic]/8 [dcl.fct]/25 Under our proposal, `vs` would be a pack of `T` parameters.

A fixed or explicitly supplied `T` permits ordinary conversions from different source types. Deduced `T` requires consistent deductions across applicable arguments. [temp.deduct.type]/2 Unlike homogeneous template packs' constant arguments, function arguments may be runtime values. Their count remains compile-time. [temp.variadic]/9

An invented constant template parameter carries that count, making the declaration an abbreviated function template whose specializations have ordinary fixed-arity function types.

As in R2, one trailing homogeneous pack covers the library use cases and lets the remaining arguments determine its length. Declaration-site lengths need new syntax and multiple packs need argument-partitioning rules so both remain deferred. Existing pack expansions retain their meanings.

Earlier revisions received EWGI support, but Belfast's 2019 EWG poll to accept P1219 for C++23 failed. [P1219-EWG-RECORD] We return with library comparisons, further evidence of programmer demand, and a revised count model.

Both named `int... values` and unnamed `int...` packs are proposed. Since Belfast, C++26 has deprecated the comma-less C-varargs spelling. [depr.ellipsis.comma] We pair mandatory commas and reclamation in C++29 so parameter names do not change pack meaning. We ask EWG to confirm that direction first.

3 Library use cases

3.1 Combine call-site diagnostics with count-dependent results

Abseil's internal `Layout<Ts...>` describes one array per template argument in one allocation. `PartialType<N>` records how many leading sizes `Partial` receives, even when their values are runtime. [ABSL-LAYOUT]

With runtime `size_t` values `int_count` and `double_count`:

```
using ArrayLayout =
    absl::container_internal::Layout<int, double, char>;

auto ints_known = ArrayLayout::Partial(int_count);
auto ints_and_doubles_known =
    ArrayLayout::Partial(int_count, double_count);
```

The calls return `ArrayLayout::PartialType<1>` and `ArrayLayout::PartialType<2>`. One supplied size permits `int` and `double` offsets and two also permit `char`:

```
auto double_offset = ints_known.Offset<double>();
auto char_offset = ints_and_doubles_known.Offset<char>();
```

`ints_known.Offset<char>()` is rejected at compile time because the preceding `double` size is missing. A replacement must preserve these count-selected result types and offsets. [ABSL-LAYOUT]

The shipped declaration deduces every argument type and casts to `size_t` in its body. A homogeneous pack would deduce only the count and initialize `size_t` parameters at the call:

Table 2: Abseil's `Partial` and a proposed homogeneous declaration. Both retain the argument-count check.

Abseil at d522178

```
template <class... Sizes>
static constexpr
 ↪  PartialType<sizeof...(Sizes)>
Partial(Sizes&&... sizes) {
  static_assert(sizeof...(Sizes) +
      StaticSizeSeq::size() <=
       ↪  sizeof...(Ts));
  return PartialType<sizeof...(Sizes)>(
      static_cast<size_t>(
        std::forward<Sizes>(sizes))...);
}
```

With homogeneous packs

```
static constexpr auto
Partial(size_t... sizes)
    -> PartialType<sizeof...(sizes)> {
  static_assert(sizeof...(sizes) +
      StaticSizeSeq::size() <=
       ↪  sizeof...(Ts));
  return PartialType<sizeof...(sizes)>(
      sizes...);
}
```

A trailing return type puts `sizes` in scope for `sizeof...(sizes)`. [basic.scope.param]/1

For `int signed_count`, Abseil's authors want warnings for `ArrayLayout::Partial(signed_count)` but not `ArrayLayout::Partial(3)`. Their comment explains that explicit casts suppress both, while implicit forwarding-body conversions can warn for both. It calls for "a variable number of parameters of a certain type, a.k.a. homogeneous function parameter packs".

Fixed `size_t` parameters expose the original argument expressions to call-site conversion diagnostics. [expr.call]/7 Ordinary initialization also rejects explicit-only conversions, such as explicit `operator size_t()`, accepted by the shipped `static_cast`. [over.match.conv]/1 [expr.static.cast]/5 A `std::convertible_to<size_t>` constraint leaves conversions in the body. [concept.convertible]/1

Warnings depend on the compiler and enabled options. Converting negative integers to unsigned remains well-formed. [conv.integral]/3

For `Layout<Ts...>` without static sizes, current C++ can preserve conversions and result types with overloads for arities zero through `sizeof...(Ts)`. Its constructor already expands an index pack through `IntToSize` for `size_t` parameters. Here `LayoutType<N, Ts...>` stands for `PartialType<N>`:

```
template <size_t>
using IntToSize = size_t;

template <class Seq, class... Ts>
struct PartialOverload;

template <size_t... Is, class... Ts>
struct PartialOverload<std::index_sequence<Is...>, Ts...> {
  static constexpr LayoutType<sizeof...(Is), Ts...>
  Partial(IntToSize<Is>... sizes) {
    return LayoutType<sizeof...(Is), Ts...>(sizes...);
  }
};

template <class Seq, class... Ts>
struct PartialOverloads;

template <size_t... Ns, class... Ts>
struct PartialOverloads<std::index_sequence<Ns...>, Ts...>
    : PartialOverload<std::make_index_sequence<Ns>, Ts...>... {
```

```
  using PartialOverload<std::make_index_sequence<Ns>, Ts...>::Partial...;
};
```

`Layout` exposes these generated overloads by inheriting `PartialOverloads<std::make_index_sequence<sizeof...(Ts) + 1>, Ts...>`. A homogeneous pack needs only one declaration.

3.2 Preserve conversion ranking without fixed-arity wrappers

CasADi's `horzcat`, `vertcat`, and `diagcat` concatenate symbolic matrices horizontally, vertically, and diagonally. At d30fbc4, fifteen positional wrappers cover arities two through six. The enclosing `SparsityInterface` specialization fixes their `const MatType&` parameters. [CASADI-CONCAT] The library call `vertcat(dx_cat, 1)` relies on scalar conversion to that type.

Table 3: CasADi's vertical concatenation wrappers (two arities shown) and a proposed replacement for the family. Both initialize references to a fixed matrix type.

CasADi at d30fbc4

```
inline friend MatType vertcat(
    const MatType& x, const MatType& y) {
  return vertcat(std::vector<MatType>{x,
    ↪ y});
}

inline friend MatType vertcat(
    const MatType& x, const MatType& y,
    const MatType& z) {
  return vertcat(std::vector<MatType>{x,
    ↪ y, z});
}
```

With homogeneous packs

```
friend MatType vertcat(
    const MatType& first,
    const MatType& second,
    const MatType&... rest) {
  return MatType::vertcat(
      std::vector<MatType>{first,
        ↪ second, rest...});
}
```

The fixed parameters retain the two-operand minimum and the trailing pack removes the six-argument ceiling, likewise for horizontal and diagonal concatenation.

A constrained deduced pack in the same class also replaces the overload family:

```
template <class... Args>
  requires (sizeof...(Args) >= 2 &&
            (std::is_convertible_v<
                const Args&, const MatType&> && ...))
friend MatType vertcat(const Args&... args) {
  return MatType::vertcat(std::vector<MatType>{args...});
}
```

The declarations differ when an argument converts to multiple matrix types. For mutable `SX symbolic` and `IM indices` (symbolic-expression and integer matrices), indexing yields `SubIndex<SX, IM>` derived from `SX`. [CASADI-PROXY] Consider these constructed calls:

```
vertcat(symbolic(indices), 1);
vertcat(1, symbolic(indices));
```

Argument-dependent lookup finds concatenation functions for `SX` and the proxy's `IM` template argument. [basic.lookup.argdep]/3-4 Fixed parameters select `SX` in both calls because base binding outranks conversion to

`IM`. [over.ics.rank]/2.1 With deduced packs, both candidates bind directly and satisfy the constraints, making both calls ambiguous. [over.match.best.general]/2 Homogeneous packs retain the fixed destinations for ranking.

Fixing only the first parameter as `const MatType&` leaves the second call ambiguous. Even two fixed parameters leave `vertcat(1, 1, symbolic(indices))` ambiguous because its distinguishing operand is in the deduced tail.

Competing `std::initializer_list<SX>` and `std::initializer_list<IM>` overloads are also ambiguous: list ranking uses the worst element conversion so the scalar's user-defined conversions erase the indexed operand's distinction. [over.ics.list]/5

Current C++ supports arbitrary counts with an explicit destination, as in `SX::vertcat({symbolic(indices), 1})`. Existing or generated wrappers preserve positional calls. A homogeneous pack would directly express two or more `const MatType&` parameters with one declaration per operation.

3.3 Broader demand for fixed-type packs

R2 includes deduced-type and fixed-type homogeneous interfaces. [P1219R2] Later requests seek fixed-type positional interfaces [REDDIT-ENUM-PACK] [SO-STRING-VIEW-CONCAT], addressing body conversions [SO-TYPED-ACCUMULATOR], ordinary conversion ranking [LOEW-HOMOGENEOUS-OVERLOADS], and by-value initialization without additional body construction. [MAZIERES-PARAM-PACK] The count would also permit constraints such as even arity. [SO-10]

The discussions offer constrained packs and fixed-parameter emulations. For these interfaces, we consider direct parameter typing easier to write, maintain, and reason about than constraints, helpers, or generated overloads. It reduces helper code and repeated declarations while retaining ordinary initialization and conversion ranking.

4 Sequence alternatives

Initializer lists and vectors accept fixed-type elements as one sequence argument, without a parameter per element.

Table 4: Homogeneous interfaces, each permitting a fixed element type. The vector column excludes `bool`.

Property	Homogeneous pack	`std::initializer_list<T>`	`std::vector<T>`
Elements and mutation	Value or reference parameters with declared cv-qualification	Const backing-array elements	Owned elements, mutable through a non-const vector
Ownership and lifetime	Values owned, references borrowed under ordinary lifetime rules	View. Copies neither copy elements nor extend backing-array lifetime	Owns storage and elements. Reference parameters borrow
Contiguous storage	No array or contiguity guarantee	Contiguous backing array	Contiguous storage
Count and iteration	Template count. Pack expansion, folds, or pack indexing	`size()` and ordinary iteration	`size()` and ordinary iteration
Runtime-sized input	No direct runtime-range expansion into arguments	No arbitrary runtime-range constructor	Accepts runtime-sized input and can grow

Property	Homogeneous pack	`std::initializer_list<T>`	`std::vector<T>`
Storage and element work	Ordinary parameter initialization, no container allocation required	Backing-array initialization	Storage construction can allocate and construct elements. Passing an existing vector by reference does neither
Call syntax	`f(a, b, c)`	`f({a, b, c})`	`f(existing)` or vector construction, such as `f({a, b, c})`
Function declarations	Template with one trailing homogeneous pack	Can be non-template and precede other parameters	Can be non-template and precede other parameters

Neither an initializer list nor a vector carries its count in its type, preventing count-selected return types. Static spans, arrays, and array references do carry it, permitting results such as Abseil's `PartialType<N>` even with runtime elements. For example:

```
template <std::size_t N>
auto span_count(std::span<const std::size_t, N>) {
    return std::integral_constant<std::size_t, N>{};
}

template <std::size_t N>
auto array_count(const std::size_t (&)[N]) {
    return std::integral_constant<std::size_t, N>{};
}

const std::size_t sizes[]{3, 5};
auto from_span = span_count(std::span{sizes});
auto from_array = array_count(sizes);
auto from_braces = array_count({3, 5});
```

All three results are `std::integral_constant<std::size_t, 2>`. Since `array_count({3, 5})` deduces a bound and initializes a temporary array, an analogous array-reference `Partial({int_count, double_count})` could return `PartialType<2>` without named storage or an explicit extent. [temp.deduct.call]/1

Braced arrays use list-initialization, rejecting narrowing accepted by ordinary `size_t` parameters in Abseil's signed-variable case. Empty braces cannot deduce the bound in `array_count({})`. [temp.deduct.call]/1 Explicit zero also fails because built-in array bounds must be positive. [dcl.array]/1 Static spans and `std::array` permit zero extent but retain grouped arguments. [span.cons]/1 A homogeneous `size_t` pack permits both the conversion and zero arguments positionally.

A span over an existing array avoids constructing another array but requires matching element types apart from permitted qualification conversions. Thus an `int` array cannot initialize `std::span<const std::size_t>`, although `size_t` parameters could accept its individual elements. [span.cons]/13

Callers can convert elements by constructing an explicitly typed `std::array<T, N>` or vector from braces. A const-element span also accepts a temporary array through double braces such as `{{1, 2}}`. The temporary array lasts through the call's full-expression. [class.temporary]/6.10 Both approaches retain list-initialization restrictions.

A container's element type fixes the conversion destination before the call. Homogeneous packs would preserve CasADi's selection by per-argument conversion ranking, without requiring callers to choose a matrix type or group arguments.

A constrained `std::ranges::input_range` suits sequence input, especially runtime-sized or non-contiguous sequences. The concept supplies no compile-time count, though deduction may retain one from an array or static-span type. Convertibility constraints neither perform conversions nor provide separate parameters for the original arguments

so they do not reproduce Abseil's call-site diagnostics.

5 Design decisions

5.1 Definitions

Each *homogeneous function parameter pack* (*homogeneous pack*) gets an invented constant template parameter for its length.

Its *parameter pattern* is the declaration without the declarator ellipsis. [temp.variadic]/6.1 After placeholder processing invents a type parameter pack for `auto...` [dcl.fct]/22, classification occurs before enclosing-template substitution: a pattern with no unexpanded pack is homogeneous. Ordinary expansions remain expansions even when all resulting types coincide.

An occurrence is unexpanded unless inside a nested expansion's pattern. Indexing consumes the indexed pack name, leaving unexpanded packs in the index for an enclosing expansion. Neither `decltype` nor ordinary `sizeof` consumes packs. This follows existing expansion boundaries [temp.variadic]/6-8 and constraint rules [temp.constr.fold].

In the following table, `xs` can denote a constant template parameter pack, function parameter pack, init-capture pack, or structured binding pack.

Table 5: Parameter-pack classification under this proposal.

Parameter declaration	Unexpanded packs	Meaning under the proposal
`int... values`	Nothing	Homogeneous `int` pack
`T... values`	Nothing	Homogeneous `T` pack
`auto... values`	Invented type pack	Ordinary expansion
`Ts... values`	`Ts`	Expands `Ts`
`Templates<T>... values`	`Templates`	Expands `Templates`
`decltype(xs)... values`	`xs`	Expands `xs`
`decltype(sizeof(xs))... values`	`xs`	Expands `xs` despite identical resulting types
`Box<Ts...>... values`	Nothing	Homogeneous resulting `Box` type. Inner expansion consumes `Ts`
`Ts...[I]... values`	Nothing	Homogeneous pack of the selected type
`Ts...[Is]... values`	`Is`	Expands `Is`. Indexing consumes `Ts`
`Templates...[I]<T>... values`	Nothing	Homogeneous pack of the selected template's specialization
`Templates...[Is]<T>... values`	`Is`	Expands `Is`. Indexing consumes `Templates`
`decltype(xs...[I])... values`	Nothing	Homogeneous pack using the selected element's `decltype`
`decltype(xs...[Is])... values`	`Is`	Expands `Is`. Indexing consumes `xs`
`decltype(xs...[I] + xs)... values`	The second `xs`	Expands the second `xs`. Indexing consumes only the first
`decltype((xs + ...))... values`	Nothing	Homogeneous pack of the fold's result type

Parameter declaration	Unexpanded packs	Meaning under the proposal
`decltype(sizeof...(xs))... values`	Nothing	Homogeneous pack of the `sizeof...` result type
`int... values[Is]`	`Is`	Expands `Is` despite array parameter adjustment to `int*`
`decltype(requires { requires (C<Ts> && ...); })... values`	Nothing	Homogeneous `bool` pack. Nested fold-expression consumes `Ts`

Each homogeneous case has its own count. Even at zero, substitution requires valid types, indices, folds, and array bounds. The last row's nested requirement determines the `bool` requires-expression's value without constraining the function.

5.2 Pack declarations

The same declarator syntax covers ordinary expansions and homogeneous packs, whose patterns need not mention template parameters:

```
// heterogeneous function parameter pack
template <class... T>
void f(T... v);

// homogeneous function parameter pack
template <class T>
void f(T... v);

// homogeneous function parameter pack
void f(int... v);
```

Since C++20, function templates needing no template parameter names can use abbreviated syntax with parameter placeholders such as `auto`. [P1141R2] [dcl.fct]/22 Likewise, a homogeneous pack makes its declaration an abbreviated function template before body parsing, omitting the *template-head* when no template parameters are explicit.

Its invented, unnamed constant template parameter replaces R2's hidden, deduction-only length component of specialization identity. [P1219R2] Ordinary template arguments permit explicit counts, target-free specialization selection for address-taking or callable-deducing interfaces, and ordinary template-argument identity.

The count has type `std::size_t`, as does `sizeof...`. The wording uses the type of `sizeof` to avoid requiring a declaration or header. [expr.sizeof]/5

A specialization's count is visible in `std::meta::template_arguments_of`. [meta.reflection.queries]/59 For named packs, `sizeof...(values)` exposes it within the parameter's scope.

As function templates, these declarations cannot be virtual, block-scope, or members of local non-closure classes. [temp.mem]/2-4 [dcl.fct]/24 Definitions must meet template reachability requirements, except for explicitly instantiated specializations. [temp.pre]/10 Replacing non-template overloads requires checking these restrictions, ABI compatibility, and overload ordering.

[P1061R10] section 3.3 removed non-template structured binding packs after objections to R9's implicit template region. That region covered an ordinary function's remainder after the binding declaration. Here, the signature declares the whole function a template.

A homogeneous pack must be the only function parameter pack, appear last, and have no following C ellipsis. Preceding non-pack parameters leave the remaining arguments to determine its count without partitioning. Its parameters otherwise behave like any function parameter pack's.

These declarations violate the placement rules:

```
void misplaced(int... values, int last);
void two_packs(int... left, double... right);
void trailing_varargs(int... values, ...);
```

The pack must declare the function's or lambda call operator's own parameters. Thus `using Callback = void(int... values);` and `void register_callback(void (*callback)(int... values));` remain ill-formed: neither declares a function template to own the count. These forms need separate count-introduction and deduction rules. Ordinary expansions in aliases and nested function types stay unchanged. The exception is a captureless generic lambda's conversion template, which shares the call operator's count.

```
void named_f(int... values);
auto* named_pf = &named_f<3>;

template <class T>
void named_g(T... values);
auto* named_pg = &named_g<int, 3>;
```

Both pointers have type `void (*)(int, int, int)`. The homogeneous declarations may omit parameter names.

The count follows all explicit template parameters and invented `auto` type parameters without changing their positions:

```
template <class Tag>
void tagged(auto tag, int... values);
auto* tagged_pair = &tagged<void, char, 2>;
```

The arguments supply `Tag`, the invented `tag` type, then count, giving `void (*)(char, int, int)`. Lambda call operators use the same order.

The trailing-pack rule keeps invented parameters in function-parameter order. Non-trailing support would require revisiting this.

[Drafting note: Explicit-argument assignment remains unspecified for preceding template parameter packs, including constant packs and `auto` parameters. The count stays last.]

5.2.1 Alternatives

We retain R2's rejection of a bare `template` introducer and empty template heads. [P1219R2] Empty `template <>` introduces an explicit specialization and cannot introduce a homogeneous pack. [temp.expl.spec]/1 The candidate wording states this explicitly, because the working draft lacks the corresponding `auto` rule.

In `int auto... values`, `auto` has no type to deduce and would only mark template status. `int` is not a concept constraint either. We reject this redundant marker and retain symmetry with homogeneous template packs such as `std::size_t... values`.

R2 considered a `template <...>` marker for homogeneous packs [P1219R2], permitting unnamed packs without reclaiming comma-less C varargs. The lambda spelling would be `[]<...>(int... values) {}`.

Kona's discussion objected to another template introducer without a *template-head*. [P1219-EWG-RECORD] The abbreviated-template model adds no token: an ellipsis before the name already declares packs in `Ts... values` and `auto... values`. The invented count supplies the template parameter as `auto`'s invented type parameter does. R2 reports that Richard Smith suggested the marker, which he and David Vandevoorde thought simpler to implement across compilers. Smith later agreed by e-mail that implementations supporting P1141R2's headless declarations should handle homogeneous packs without added syntax. [P1219R2] We prefer the existing cue.

The invented count covers the present use cases, though declaration-site lengths would let authors name and position it. We defer these lengths and multiple or non-trailing homogeneous packs, which need count association and deduction rules. Poll 4 (Proposed EWG polls) asks EWG to confirm this scope. [P2347R2] proposes deducing a single non-trailing pack's length from argument count and required trailing parameters.

R2's bracketed length spelling conflicts with C++26 pack indexing: `Ts...[Len]` already selects one type. [P1219R2] [dcl.type.pack.index] A marker cannot resolve this conflict, but other spellings remain possible. Restoring abstract array-pack grammar remains separate.

5.2.2 Type deduction

In these proposed functions, `sum` fixes the element type to `int` while `total` uses the ordinary template parameter `T`:

```
int sum(int... values) {
    return (0 + ... + values);
}

template <class T>
T total(T... values) {
    return (T{} + ... + values);
}
```

`total` requires consistent `T` deductions from all applicable arguments, not the first argument's type or a common type. [temp.deduct.type]/2 Fixed types in `sum` or `total<int>` allow ordinary conversions. Explicit `T` leaves count deduction unchanged, while `sum<2>(1, 2.5)` and `total<int, 2>(1, 2.5)` fix the count.

Table 6: Proposed calls. Counts are omitted and deduced from arity.

Element type	Call	Proposed outcome
Fixed `int`	`sum()`	Length zero. Initialized addition fold returns zero.
	`sum(1, 2.5)`	Length two. Ordinary conversions initialize both parameters.
Deduced `T`	`total(1, 2)`	Deduces `T` as `int`, length two.
	`total(1, 2.5)`	Reject conflicting deductions for `T`.
	`total()`	Reject: no argument to deduce `T`.
Explicit `int`	`total<int>(1, 2.5)`	Length two, explicit type and ordinary conversions.
	`total<int>()`	Length zero, known element type.

Specialization identity comes from the template and its arguments, including the count converted to its parameter type. With a fixed element type, source argument types do not affect identity: `sum(1, 2)`, `sum(1.0, 2.0)`, and `sum<2>(1, 2)` all select `sum<2>`.

The template-argument component of [defns.signature.spec] and [defns.signature.member.spec] covers the invented count without a signature change or separate pack-length component.

Different template arguments can produce identical adjusted function types, which alone do not identify specializations. [temp.deduct.general]/3-4

An explicit count must equal the remaining argument count: `sum<3>(1, 2, 3)` and `sum<0>()` are valid, but `sum<3>(1, 2)`, `sum<2>(1, 2, 3)`, and `sum<0>(1)` are rejected. Ordinary constant template argument rules apply to the invented unsigned type. `sum<>()` deduces zero.

At zero count, the element type must be explicit, deduced elsewhere, or defaulted. Its pattern still undergoes substitution.

Ordinary reference binding and parameter adjustment apply, as do list-initialization and narrowing checks for braces. Temporaries bound to reference parameters last through the call's full-expression. [class.temporary]/6.10 Packs change neither that lifetime nor argument-initialization evaluation order. [expr.call]/7-8

5.2.3 Target types for initialization

A fixed parameter type supplies a target for braced initializers and overloaded function names in these proposed declarations:

```
struct Point {
    int x;
    int y;
};

void plot(const Point&... points);

using Step = void (*)();
void start();
template <class T> void start(T mode);
void stop();

void run_all(Step... steps);
```

Table 7: Target typing under the proposed parameter-initialization rules.

Call	Proposed outcome
`plot({0, 0}, {4, 5})`	Each initializes a temporary `Point` bound to a const reference parameter.
`plot({2.5, 3})`	Reject: list-initialization narrows `Point::x`.
`run_all(start, stop)`	The `Step` target selects `void start()`.

`auto...` cannot deduce from bare braces or the `start` overload set containing a function template. [temp.deduct.type]/5.8-5.9 Current C++ supplies targets through `std::initializer_list<Point>` and `std::initializer_list<Step>`, with calls `plot({{0, 0}, {4, 5}})` and `run_all({start, stop})`. Deduced packs instead accept explicit `Point` objects and `static_cast<Step>(start)`. [over.over]/1-3

5.2.4 Address-taking

When taking `sum` or `total` addresses, the target function type supplies the count. A length-two `sum` specialization has type `int(int, int)`.

Table 8: Proposed address-taking and declarations. Rows are independent. Element type and count must match the identified function.

Operation	Proposed outcome
`int (*zero)() = sum`	Select length zero.
`int (*one)(int) = sum<>` `int (*two)(int, int) = sum<>`	Target types select different specializations.
`auto two = &sum` `auto two = &sum<>`	Reject: no length determined.
`auto two = &total<int>`	Reject: explicit `T` supplies no length.
`auto two = &sum<2>` `auto two = &total<int, 2>`	Select two-parameter specialization without a target type.

Operation	Proposed outcome
`int (*two)(int, int) = sum<3>`	Reject: explicit length mismatches target.
`int (*two)(double, double) = sum`	Reject: parameter-type mismatch. Call conversions cannot adapt function pointer signatures.
`int sum(int...)` and the named definition	Same template.
`int sum(int, int)`	Ordinary overload, not an implicit specialization declaration.

Corresponding homogeneous packs require corresponding parameter patterns and invented count parameters in the same template position. Parameter names do not distinguish declarations. Matching specialization function types alone do not make ordinary expansions correspond to homogeneous packs.

The bare name `sum` supplies no target type to callable-deducing interfaces, preventing deduction in `std::thread(sum, 1, 2)`, `std::invoke(sum, 1, 2)`, and `std::function<int(int, int)> callback = sum`. [temp.deduct.type]/5.8.3 Each can use `sum<2>`, a pointer initialized with a matching target type, or a forwarding lambda. Likewise, only `decltype(sum<2>)`, not `decltype(sum)`, identifies a specialization.

Existing explicit-specialization and explicit-instantiation syntax remains. Here, omitted counts are deduced from ordinary parameter lists:

```
template <>
int sum<>(int value) {
    return value + 10;
}

template int sum<>(int, int);
```

Explicit `template <> int sum<1>(int value)` and `template int sum<2>(int, int)` identify the same specialization and instantiation. The counts must match their parameter lists.

5.2.5 Overload ordering

Ordinary conversion ranking, partial ordering, constraint subsumption, and element-type comparisons remain. Unordered candidates stay ambiguous, without preference for homogeneous syntax. Partial ordering compares declarations, preserving pack origins [temp.deduct.partial]/8, rather than only resulting fixed-arity specializations. [temp.func.order]/3-6

Calls without pack arguments contribute no element-type comparisons. Default template arguments can establish viability without fixing dependent patterns for ordering.

Table 9: Proposed overloads under ordinary ordering. Sets are independent.

Overloads	Call	Proposed outcome
`void fixed(int... values);` `template <class T> void fixed(T... values);`	`fixed(1, 2)`	Select the fixed-element template.
	`fixed(1.0, 2.0)`	Select the deduced-element template by better conversions.
	`fixed()`	Select fixed-element template: `T` cannot be deduced.

Overloads	Call	Proposed outcome
`template <class T> void packed(T... values);`	`packed(1, 2)`	Ambiguous.
`template <class... Ts> void packed(Ts... values);`	`packed(1, 2.0)`	Select heterogeneous template: homogeneous `T` deductions conflict.
	`packed()`	Select heterogeneous template: homogeneous `T` cannot be deduced.
`void empty(int... values);` `void empty(double... values);`	`empty()`	Ambiguous: zero count supplies no element-type comparison.
`void pair(int, int);` `void pair(int... values);`	`pair(1, 2)`	Select the non-template overload after tied conversions.
`template <class T> void paired(T, T);` `template <class T> void paired(T... values);`	`paired(1, 2)`	Select fixed-arity template by ordinary partial ordering.
`template <class T> void single(T);` `void single(int... values);`	`single(1)`	Ambiguous: neither the fixed-arity pattern nor fixed element type makes one template more specialized.

For `packed(1, 2)` in Table 9, deduction between transformed pack patterns succeeds both ways. Both declarations have trailing packs so [temp.deduct.partial]/11 prefers neither. Adding the deduced homogeneous overload to the heterogeneous catch-all therefore makes same-type calls ambiguous. A fixed element type can still establish ordering. The candidate wording adds no preference. We would accept a tie-break preferring a homogeneous trailing pack over an ordinary one for equal-length calls, allowing this addition to existing `Ts...` overload sets, and ask EWG to decide in Poll 8 (Proposed EWG polls).

Replacing constrained emulation can also change overload selection. Beside `template <class T> void single(T)`, `void single(std::same_as<int> auto...)` loses for `single(1)`. Replacing it with `void single(int...)` produces Table 9's ambiguity. [temp.deduct.partial]/8

Length-dependent associated constraints require count determination before satisfaction substitutes ordinary template arguments, including counts and enclosing specializations' arguments, without separate count bindings. Rechecking the same atomic constraint with the same arguments retains the existing consistency requirement. [temp.constr.atomic]/3

The count adds no atomic-identity or subsumption test, including no arithmetic implication between separate `sizeof...(values) >= 1` and `sizeof...(values) >= 2` predicates. [temp.constr.atomic]/2 [temp.constr.order]/1.2

[Drafting note: The rules for matching corresponding packs in atomic and fold-expanded constraints remain incomplete.]

5.2.6 Constructors

If supported, homogeneous constructors would be implicit templates in ordinary overload selection. [over.match.best.general]/2.4 Initializer-list constructors, however, have priority under [over.match.list]/1.

Table 10: Conditional comparison of initializer-list and homogeneous constructors.

Current C++	If homogeneous constructors are supported

`struct Values {`
`  Values(std::initializer_list<int>);`
`  explicit Values(const int&, const int&);`
`};`
	`struct Values {`
	`  Values(const int&... elems);`
	`  explicit Values(const int&, const int&);`
	`};`

Table 11: Constructor selection under ordinary ranking.

Declaration	With initializer list	With homogeneous pack
`Values values{1, 2, 3};`	Selects the list constructor.	Selects the homogeneous constructor.
`Values values{1, 2};`	Selects the list constructor.	Selects the explicit two-argument constructor.
`Values values = {1, 2};`	Selects the list constructor.	Rejected: selected two-argument constructor is explicit.
`Values values = 5;`	Rejected: no converting constructor from `int`.	Selects the non-explicit specialization with count one.
`Values values;`	Rejected: no default constructor.	Selects the specialization with count zero.

[Drafting note: The candidate wording includes constructors subject to Poll 7, but support remains undecided. It excludes deduction guides, with support undecided pending analysis of deduction and return types. For class templates, [over.match.class.deduct]/1.1 forms implicit deduction candidates from constructor template parameters and parameter lists, including the invented count. Whether implicit guides from homogeneous constructors fall under that exclusion or satisfy the context restriction remains unresolved by Poll 7. The examples still need validation treating homogeneous constructors as implicit templates.]

5.3 Lambdas

Lambdas use the same count model. This proposed call operator remains a template despite its fixed element type:

```
auto product = [](double... factors) {
    return (1.0 * ... * factors);
};
```

Here, `product()` and `product(0.5, 0.8)` return `1.0` and `0.4`. The count lets `product.operator()<2>(0.5, 0.8)` select the specialization with count two. A current-language `std::same_as<double> auto... factors` lambda accepts both ordinary calls but takes element types as template arguments.

For `double (*multiply)(double, double) = product`, the captureless lambda's generic conversion function template deduces count two from the target. Unary `+product` lacks a target and count and is rejected. Conversion remains available only without a *lambda-capture* or explicit object parameter. [expr.prim.lambda.closure]/12-14

An enclosing pack's ordinary expansion, including indexed forms, needs no independent count and alone does not make a lambda generic. [expr.prim.lambda.general]/6

Homogeneous pack elements would follow ordinary capture and lifetime rules for function parameters.

5.4 Pack indexing

Homogeneous packs would use existing expansion, folding, and constant-index access. Here, `select_sample` indexes with a template argument:

```
template <std::size_t I>
double select_sample(double... samples)
    requires (I < sizeof...(samples)) {
    return samples...[I];
}
```

With runtime `double reading`, `select_sample<1>(reading, 0.5, 1.0)` selects the second sample using the supplied index and deduced length. Index 3 fails the constraint. A `std::same_as<double> auto... samples` pack gives the same results. [expr.prim.pack.index]/2-4

In parameter patterns, `Ts...[1]` selects the second type. As in Definitions, `Ts...[Is]` consumes `Ts`, leaving `Is` for enclosing expansion. [temp.variadic]/6.15 [dcl.type.pack.index]/2-4 The index pack can differ in length from `Ts` and repeat types.

Rows in Table 12 are independent, with enclosing declarations of `Ts` and `Is` and valid selected indices.

Table 12: Indexed lambda parameters. A scalar index permits a new homogeneous count. An index pack produces an ordinary expansion.

Spelling	C++26 interpretation	Proposed interpretation
`[](Ts...[1]... values) {}`	Ill-formed. No unexpanded pack.	A homogeneous pack of the selected type with its own count.
`[](Ts...[Is]... values) {}`	The second ellipsis expands `Is` into one parameter list.	Unchanged. No new count.

With a valid selected type, unnamed `Ts...[1]...` denotes one parameter followed by C varargs in C++26. Indexing consumes `Ts` so the final ellipsis follows [dcl.fct]/26's C-varargs branch. Reclamation changes it to a homogeneous pack. Inserting a comma, as in `Ts...[1], ...`, preserves C varargs.

Scalar indexing fixes the proposed pack's type before count deduction:

```
template <std::size_t I, class... Ts>
auto selected_counter() {
    return [](Ts...[I]... values) {
        return sizeof...(values);
    };
}

auto count_doubles = selected_counter<1, int, double, char>();
```

`count_doubles(0.5, 1.0)` returns 2: scalar indexing selects `double` from three types before the call independently deduces its count.

An empty outer `Is` expansion need not instantiate indexed selection. [temp.variadic]/9 A fixed index must still designate an element whenever its type is required, including homogeneous lambda declarations with zero-argument calls. [dcl.type.pack.index]/2

5.4.1 Array parameters

In C++26, `int... values[Is]` expands `Is` into array bounds, leaving none for enclosing folding. Positive extents 1, 2, and 3 adjust to three `int*` parameters. Zero extents are independently invalid. [dcl.array]/1

C++26 [dcl.name]/1 has no array-suffix production for abstract (unnamed) pack declarators. P2662R3 removed it with pack indexing. [P2662R3]

The proposal retains this grammar. An array alias could supply an unnamed homogeneous parameter type with ordinary adjustment and an extent independent of its count. Parenthesized unnamed pack declarators (CWG1488) remain out of scope because aliases need no new declarator form.

6 Reclaiming the deprecated comma-less spelling

The C++29 design requires a comma between a parameter list and C varargs and reclaims the comma-less spelling for unnamed homogeneous packs. Otherwise, `void f(int... args)` would declare a homogeneous pack while `void f(int...)` retained deprecated C varargs so adding or removing a name would change meaning and template status. The polls assume this pairing. Reclamation also permits omitting unused names as `void g(auto...)` and `template <class... Ts> void h(Ts...)` already do. [depr.ellipsis.comma]

James Touton's [P0281R0] proposed removing comma elision in 2016 but failed because of its impact on existing code ([P3176R1], section 2). Kona's 2019 EWGI vote favored mandatory commas by 10/6/1/0/0 without establishing a C++29 reclamation timetable. [P1219-EWG-RECORD] Jan Schultke's [P3176R1], *The Oxford variadic comma*, chose deprecation: no proposed feature required removal, the earlier attempt had failed, and the motivation did not justify abruptly breaking code (section 6.1). This proposal requires removal.

One standard cycle separates C++26 deprecation from proposed C++29 reclamation. Adding a comma preserves C varargs in every language mode, permitting migration before C++29. GCC enables `-Wdeprecated-variadic-comma-omission` by default in C++26 mode. [GCC-CXX-DIALECT] Clang enables `-Wdeprecated-missing-comma-variadic-parameter` by default and reports P3176R1 implemented. [CLANG-DIAGNOSTICS] [CLANG-20-NOTES] R2's ACTCD19 header survey found 6,318 comma-less C-varargs lines: 6,182 in Boost copies (879 after deduplication) and 136 elsewhere. [P1219R2] This predates deprecation and does not measure remaining migration cost in maintained and vendored code.

A one-cycle precedent is [P1161R3]'s C++20 deprecation of unparenthesized comma subscripts and [P2128R6]'s C++23 replacement. P2128R6 deliberately withheld multidimensional meaning from C arrays and existing `valarray` interfaces, making deprecated expressions ill-formed before any later reinterpretation. It cannot establish that immediate reinterpretation of comma-less varargs is harmless.

A deprecated subscript could acquire new meaning silently inside any expression. For comma-less declarations, the diagnostics above can locate the comma insertion preserving meaning in every language mode. Unmigrated declarations can still change meaning without required diagnostics, as below.

6.1 Current rules

A *varargs ellipsis* denotes C varargs, not a parameter pack. C23 and C++26 permit `...` without preceding parameters. [N3096] C++26 also permits but deprecates omitting the comma between a parameter list and C varargs. [depr.ellipsis.comma]

A final ellipsis that could belong to an abstract (unnamed) declarator declares a pack if its type names an unexpanded template parameter pack or contains `auto`, otherwise C varargs. [dcl.fct]/26 Writing `basic_string_view<Char, Traits> svs ...` to declare a pack gives C varargs today. [SO-STRING-VIEW-CONCAT]

These C++26 declarations therefore differ:

```
// declares a variadic function template with a function parameter pack
template <class... T>
```

```
void f(T...);

// same as void f(T, ...)
template <class T>
void f(T...);
```

6.2 Proposed change

Requiring the comma and removing unnamed disambiguation lets comma-less declarations introduce homogeneous packs. Standalone `...`, comma-separated C varargs, and ordinary pack expansions retain their meanings.

Table 13: Independent declarations before and after proposed C++29 reclamation. The middle rows are namespace-scope declarations. In the last row, `Ts` is an existing enclosing template type parameter pack.

C++26 declaration	After reclamation	Migration preserving C varargs
`void f(int value...)`	Ill-formed. The name precedes the ellipsis	`void f(int value, ...)`
`void f(int...)`	Homogeneous pack with fixed element type int	`void f(int, ...)`
`template<class T> void f(T...)`	Homogeneous pack with element type T	`template<class T> void f(T, ...)`
`void f(Ts......)`	Ill-formed. The final C ellipsis lacks a comma	`void f(Ts..., ...)`

Table 13's first and last rows become ill-formed, requiring diagnostics. The middle rows stay valid at namespace scope without required diagnostics for changed meanings. Reclaimed `void f(int...)` becomes ill-formed at block scope, as a virtual member, or in a local non-closure class under existing template restrictions. [dcl.fct]/24 [temp.mem]/2-3 The third row already has these restrictions.

The six-dot declaration `template<class... Ts> struct S { static void f(Ts......); };` is valid today: `S<int>::f(1, 2.0)` passes its second argument through C varargs. Migration must insert a comma between the ellipses, including in named expansions: `Ts... values...` becomes `Ts... values, ...`. Removing the last ellipsis would change the interface. [P3176R1]

An old `f(1, 2.5)` call to `void f(int...)` passed a double through C varargs. Reclamation instead initializes two int parameters, using an ordinary floating-integral conversion. It also permits empty calls and produces fixed-count specializations without C ellipsis. Even successful recompilation thus leaves a source and potential link compatibility break. Adoption includes approving these known source and link incompatibilities under [SD10], which Poll 2 (Proposed EWG polls) asks separately.

6.2.1 Migrating definitions and function types

Legacy definitions using `va_start` require comma insertion to preserve the C ellipsis that homogeneous parameters cannot supply. C++26 discards the second and subsequent macro arguments so a surviving parameter name cannot portably trigger diagnostics. [cstdarg.syn]

Inconsistently migrated declarations and definitions can validly declare separate homogeneous-template and C-varargs overloads. For `void f(int...)` and `void f(int, ...)` after reclamation, zero arguments select the template. One argument convertible to `int` selects the non-template C-varargs overload. [over.match.best.general]/2.4 With two or more arguments all convertible to `int`, the template wins because ordinary conversions outrank ellipsis conversions.

[over.ics.rank]/2.1 If an additional argument cannot initialize `int`, C varargs can remain viable. A missing selected-template definition can surface during instantiation or linking. Changing a declaration provides no link compatibility with an existing C-varargs definition.

The alias `using F = void(int...)` becomes ill-formed: function type aliases cannot introduce homogeneous function templates. It must become `using F = void(int, ...)` to retain C varargs. Reclaimed declarations no longer match C-varargs pointer targets, though fixed-arity targets can deduce a specialization's count. Passing a formerly non-template name to callable-deducing interfaces also needs a selected specialization, typed pointer, or wrapper.

The conversion changes can also affect well-formed metaprogramming probing callability, taking addresses, or recognizing C-varargs function types. Preserving C varargs requires commas across declarations, definitions, pointer and member-pointer types, traits, and macro-generated declarations. Ordinary expansions such as `template<class... Ts> void f(Ts...)` stay unchanged.

Reclamation needs grammar coordination with [P3776R1], *More trailing commas*, including between ordinary packs and C varargs. That paper proposes trailing parameter-list commas except after C ellipsis parameters.

7 Straw polls

7.1 Proposed EWG polls

We ask EWG to take the first poll before the others. The remaining polls assume it, since the design gives a pack the same meaning with and without a parameter name (Reclaiming the deprecated comma-less spelling).

1. We want C++29 to require a comma before C varargs and to reclaim the comma-less spelling for unnamed homogeneous function parameter packs.
2. We accept the source and link incompatibilities for comma-less C-varargs declarations that Poll 1 causes, as recorded in the Annex C entry.
3. We want homogeneous function parameter packs for C++29.
4. We want C++29 to permit at most one homogeneous function parameter pack per parameter list, in trailing position, deferring declaration-site lengths and multiple or non-trailing homogeneous packs.
5. We want each homogeneous function parameter pack to have an invented constant template parameter whose argument specifies the pack's length and participates in function template specialization identity.
6. We want functions with homogeneous function parameter packs to be function templates without requiring a *template-head*.
7. We want C++29 to support homogeneous function parameter packs in constructors.
8. We want a tie-break preferring a trailing homogeneous function parameter pack over an ordinary trailing function parameter pack when partial ordering would otherwise leave a call ambiguous.

7.2 Historical results

The polls in Tables 14 to 16 concern earlier revisions, before the C++26 changes addressed here. Proposed Polls 1, 3, and 6 revisit the first, second, and last rows of Table 14. The `template <>` row is the empty template-head spelling this revision rejects (Alternatives).

Table 14: Kona EWGI polls, February 2019. [P1219-EWG-RECORD]

Poll	SF	F	N	A	SA
Make vararg declaration comma mandatory.	10	6	1	0	0
This should be valid and declare a template: `template <class T> constexpr T min(T v1, T... vs);`	10	5	1	0	0

Poll	SF	F	N	A	SA
This should be valid and declare a template: `template <> void foo(int... v);`	0	4	7	2	3
This should be valid and declare a template: `void foo(int... v);`	4	6	4	0	1

Table 15: Cologne EWGI poll, July 2019, following revisions incorporating the Kona feedback. [P1219-EWG-RECORD]

Poll	SF	F	N	A	SA
Forward to EWG.	5	2	3	0	0

Table 16: Belfast EWG poll, November 2019. The public record was posted on January 22, 2020. [P1219-EWG-RECORD]

Poll	SF	F	N	A	SA
Accept P1219 as presented for C++23	2	8	8	9	2

The Belfast poll (Table 16) failed. Although Kona's public record identifies objections to template-introducing syntax and to conflicts with existing syntax, it does not establish why Belfast later failed. [P1219-EWG-RECORD]

8 Implementation experience

P1219R2 links an experimental Clang branch. [P1219R2] Its tests cover headless function and lambda declarations and the earlier `template <>` primary-template syntax, but do not establish implementation of this revision's invented count model. [P1219-CLANG-TESTS] Such an implementation may not be ready for Brazil. We seek EWG design direction.

[Drafting note: Complete and document an implementation of this revision's syntax and count model.]

9 Wording

Wording is relative to [N5054].

9.1 Lambdas

9.1.1 General [expr.prim.lambda.general]

Modify the first sentence of [expr.prim.lambda.general]/6 and append to Example 4:

6 A lambda is a generic lambda if the *lambda-expression* has any generic parameter type placeholders ([dcl.spec.auto]), ~~or~~ if the lambda has a *template-parameter-list*, or if its *parameter-declaration-clause* declares a homogeneous function parameter pack ([dcl.fct]).

[*Example 4:*

```
  auto x = [](int i, auto a) { return i; };               // OK, a generic
 ↪  lambda
  auto y = [](this auto self, int i) { return i; };       // OK, a generic
 ↪  lambda
  auto z = []<class T>(int i) { return i; };              // OK, a generic
 ↪  lambda
+ auto w = [](int... i) { return (0 + ... + i); };        // OK, a generic
 ↪  lambda
```

— *end example*]

9.1.2 Closure types [expr.prim.lambda.closure]

In [expr.prim.lambda.closure]/5, insert after the first sentence, and in /12, insert after the second sentence:

5 The closure type for a *lambda-expression* has a public inline function call operator (for a non-generic lambda) or function call operator template (for a generic lambda) ([over.call]) whose parameters and return type are those of the *lambda-expression*'s *parameter-declaration-clause* and *trailing-return-type* respectively, and whose *template-parameter-list* consists of the specified *template-parameter-list*, if any. If the *lambda-expression* declares a homogeneous function parameter pack, the function call operator template has the invented constant template parameter introduced by the abbreviated-function-template rules ([dcl.fct]). It follows any explicitly declared template parameters and invented type parameters. […]

[…]

12 For a generic lambda with no *lambda-capture* and no explicit object parameter ([dcl.fct]), the closure type has a conversion function template to pointer to function. The conversion function template has the same invented template parameter list, and the pointer to function has the same parameter types, as the function call operator template. If the function call operator template declares a homogeneous function parameter pack ([dcl.fct]), the parameter types of the pointer to function include a homogeneous function parameter pack with the same parameter pattern, whose count parameter is the conversion function template's invented count parameter. […]

9.2 Functions [dcl.fct]

Modify [dcl.fct]/22, /23, and /25 as follows:

22 An abbreviated function template is a function declaration that has one or more generic parameter type placeholders ([dcl.spec.auto]) or declares a homogeneous function parameter pack. […]

23 […] The invented *type-parameter*s are appended to the *template-parameter-list* after the explicitly declared *template-parameter*s. For an abbreviated function template that declares a homogeneous function parameter pack, one invented constant template parameter, the *count parameter* of the pack, is appended to its *template-parameter-list* after the explicitly declared template parameters and all invented type parameters. Its type is the type of a **`sizeof`** expression ([expr.sizeof]), and it has no default argument. Its value, the *pack count*, is the number of function parameters the pack introduces. No *template-head* is required when no template parameters are explicitly declared. The *declaration* in an *explicit-specialization* ([temp.expl.spec]) shall not declare a homogeneous function parameter pack.

[…]

25 […] When it is part of a *parameter-declaration-clause*, the *parameter-declaration* declares a function parameter pack ([temp.variadic]), whose *parameter pattern* is the *parameter-declaration* without the ellipsis. […] A function parameter pack whose parameter pattern contains neither an unexpanded pack nor a generic parameter type placeholder ([dcl.spec.auto]) is a *homogeneous function parameter pack*. A homogeneous function parameter pack shall be declared by the last *parameter-declaration* of a *parameter-declaration-clause* of the form *parameter-declaration-list*, and no other parameter of that clause shall be a function parameter pack. A function parameter pack is a pack expansion ([temp.variadic]) unless it is a homogeneous function parameter pack. […]

9.3 Removal and reclamation

Remove the comma-less alternative in [dcl.fct]/1, the indicated wording in /3, paragraph /26, and its disambiguation footnote:

1

parameter-declaration-clause:
    ...
    parameter-declaration-list$_{opt}$
    parameter-declaration-list , ...
~~   *parameter-declaration-list* ...~~

[…]

3 […] ~~Where syntactically correct and where "..." is not part of an *abstract-declarator*, "..." is synonymous with ", ...". A *parameter-declaration-clause* of the form *parameter-declaration-list* ... is deprecated ([depr.ellipsis.comma]).~~ […]

[…]

26 ~~There is a syntactic ambiguity when an ellipsis occurs at the end of a *parameter-declaration-clause* without a preceding comma. In this case, the ellipsis is parsed as part of the *abstract-declarator* if the type of the parameter either names a template parameter pack that has not been expanded or contains `auto`; otherwise, it is parsed as part of the *parameter-declaration-clause*.~~

69) ~~One can explicitly disambiguate the parse either by introducing a comma (so the ellipsis will be parsed as part of the *parameter-declaration-clause*) or by introducing a name for the parameter (so the ellipsis will be parsed as part of the *declarator-id*).~~

Remove [depr.ellipsis.comma]:

(D.5) ~~**Non-comma-separated ellipsis parameters [depr.ellipsis.comma]**~~

~~A *parameter-declaration-clause* of the form *parameter-declaration-list* ... is deprecated ([dcl.fct]).~~

~~[*Example 1:*~~

```
void f(int...);        // deprecated
void g(auto...);       // OK, declares a function
  ↪ parameter pack
void h(auto......);    // deprecated
```

~~— *end example*]~~

A terminal abstract-declarator ellipsis then belongs to the parameter declaration under homogeneous-pack or existing ordinary expansion rules.

9.4 Address of an overload set [over.over]

Insert after [over.over]/3:

3 The specialization, if any, generated by template argument deduction ([temp.over], [temp.deduct.funcaddr], [temp.arg.explicit]) for each function template named is added to the set of selected functions considered.

* For a function template that declares a homogeneous function parameter pack, an omitted argument for its invented count parameter is deduced from the target function type as specified in [temp.deduct.type]. A specialization is formed only if this argument is supplied explicitly or determined by deduction. A count mismatch causes template argument deduction to fail.

9.5 Variadic templates [temp.variadic]

Modify Example 2 in [temp.variadic]/2. Modify the second sentence of /5 and insert immediately after it, replace /6.1, modify /8 and /9.2, and replace the example in /15 as follows:

Retain the remaining requirements of paragraph 8, the other expansion patterns in paragraph 6, and the other element interpretations in paragraph 9.

2 A function parameter pack is a function parameter that accepts zero or more function arguments.

[*Example 2:*

```
  template<class ... Types> void f(Types ... args);
+ void g(int ... args);            // args is a homogeneous function parameter
 ↪  pack

  f();                              // args contains no arguments
  f(1);                             // args contains one argument: int
  f(2, 1.0);                       // args contains two arguments: int and double
+ g();                              // pack count is zero
+ g(2, 1.0);                       // pack count is two and 1.0 converts to int
+ g<1>(2, 1.0);                    // error: explicit pack count does not match
 ↪  two arguments
```

— *end example*]

[…]

5 […] The number of elements of a template parameter pack or a function parameter pack other than a homogeneous function parameter pack is the number of arguments provided for the parameter pack. The number of elements of a homogeneous function parameter pack is its pack count ([dcl.fct]). […]

[…]

(6.1) — In a function parameter pack that is a pack expansion ([dcl.fct]); the pattern is ~~the *parameter-declaration* without the ellipsis~~its parameter pattern.

[…]

8 ~~A pack whose~~An occurrence of a name ~~appears~~denoting a pack is unexpanded in a construct unless it is within the pattern of a pack expansion ~~is expanded by that pack expansion~~contained in that construct. A pack is unexpanded in a construct if the construct contains at least one such occurrence. An occurrence that declares a pack is not considered when determining unexpanded packs.

~~An appearance of the name of~~Each occurrence of a name denoting a pack within the pattern of a pack expansion is ~~only~~ expanded by the innermost ~~enclosing~~ pack expansion whose pattern contains

that occurrence. The pattern of a pack expansion shall ~~name~~contain one or more ~~packs that are not expanded by a nested pack expansion; such packs are called~~ packs that are unexpanded ~~packs~~ in ~~the~~that pattern.

All of the packs expanded by a pack expansion shall have the same number of arguments specified. An appearance of a name of a pack that is not expanded is ill-formed. […]

[…]

(9.2) — if the pack is a function parameter pack, the element is an *id-expression* designating the ith function parameter that resulted from instantiation of the function parameter pack declaration or, for a homogeneous function parameter pack, the ith function parameter it introduces;

[…]

[*Example 7:*

```
- template<typename ...Args>
- bool all(Args ...args) { return (... && args); }
+ bool all(bool ...args) { return (... && args); }

  bool b = all(true, true, true, false);
```

Within the instantiation of `all`, the returned expression expands to `((true && true) && true) && false`, which evaluates to `false`. — *end example*]

9.6 Dependency

A homogeneous-pack function type depends on its invented count until determined, even with a non-dependent element type. Existing rules make function types containing function parameter packs dependent and `sizeof...` value-dependent. [temp.dep.type]/10.7 [temp.dep.constexpr]/4 Count substitution follows ordinary template rules. Declaring the pack does not make unrelated expressions dependent.

[Drafting note: [temp.dep.expr]/3.6's pack-specific type-dependence rule covers structured binding packs, not homogeneous function parameter packs with fixed element types and undetermined counts. Remaining dependency and substitution-failure wording must address pack expressions, indexing, and count constraints, including invalid indexed element types at count zero.]

9.7 Deduction

9.7.1 General [temp.deduct.general]

[Drafting note: Substitution-order and failure-handling wording is incomplete, particularly with template parameter packs preceding the count. Explicit counts should apply before parameter matching and omitted counts be deduced before checking constraints using them.]

In [temp.deduct.general]/1, insert before Example 1, append to Example 1, and add a new example immediately after it as follows:

1 When a function template specialization is referenced, all of the template arguments shall have values. The values can be explicitly specified or, in some cases, be deduced from the use or obtained from default *template-argument*s. For a function template that declares a homogeneous function parameter pack, the argument for its count parameter shall be determined. The argument can be explicitly specified or deduced as described in [temp.deduct.call] and [temp.deduct.type]. After substitution, the pack introduces the number of function parameters specified by that argument, each formed from its parameter pattern. The parameter pattern undergoes substitution even when that number is zero.

Explicit specification and deduction of equivalent count arguments identify the same specialization under the ordinary template-argument equivalence rules. […]

[*Example 1:*

```
void f(Array<dcomplex>& cv, Array<int>& ci) {
  sort(cv);                     // calls sort(Array<dcomplex>&)
  sort(ci);                     // calls sort(Array<int>&)
}
```

and

```
  void g(double d) {
    int i = convert<int>(d);      // calls convert<int,double>(double)
    int c = convert<char>(d);     // calls convert<char,double>(double)
  }
+
+ int sum(int... n) {
+   return (0 + ... + n);
+ }
+
+ void h(int x, int y, int z) {
+   int i = sum(x, y, z);         // selects sum<3>
+ }
```

— *end example*]

[*Example 2:*

```
template <class T>
void f(T... v);

// error: homogeneous pack count is undetermined
auto p = &f<int>;

// OK, q has type void (*)(int, int, int)
auto q = &f<int, 3>;
```

— *end example*]

9.7.2 Deducing template arguments from a function call [temp.deduct.call]

In [temp.deduct.call]/1, make the inline change and insert before Example 2:

1 […] For a function parameter pack that occurs at the end of the *parameter-declaration-list*, deduction is performed for each remaining argument of the call, taking the type *P* of the *declarator-id* of the function parameter pack as the corresponding function template parameter type. Each deduction deduces template arguments for subsequent positions in the template parameter packs (if any) expanded by the function parameter pack. When a function parameter pack appears in a non-deduced context ([temp.deduct.type]), the type of that pack is never deduced. For a trailing homogeneous function parameter pack whose count argument has not been explicitly specified, deduce that argument as the number of call arguments remaining after those associated with the preceding parameters. If an explicitly supplied count differs from that number, template argument deduction fails. Deduction from the parameter pattern is performed independently for each corresponding argument for template parameters whose arguments have not been explicitly specified. Deductions for the same template parameter, including those from preceding function parameters, are combined under the ordinary deduction rules. If no call arguments remain, deduce an omitted count as zero and make no element-type deductions from the pack. […]

Retain the existing [temp.deduct.call] example unchanged.

9.7.3 Deducing template arguments from a type [temp.deduct.type]

In [temp.deduct.type]/10, make the inline change and insert immediately after the sentence shown below:

10 […] Each comparison deduces template arguments for subsequent positions in the template parameter packs (if any) expanded by the function parameter pack. When a function parameter list containing a trailing homogeneous function parameter pack is matched against a function parameter list of known length, deduce an omitted argument for the count parameter as the number of parameters remaining after matching the preceding parameters. If an explicitly supplied count differs from that number, template argument deduction fails. Match the parameter pattern against each remaining parameter using the applicable deduction rules. If no parameters remain, deduce an omitted count as zero and make no element-type deductions from the pack. This deduction applies when deducing template arguments from a function type, including the contexts described in [temp.deduct.funcaddr], [temp.deduct.decl], and [temp.deduct.conv]. It does not apply call-argument conversions. In the absence of an explicit count and a function type from which to deduce it, the count remains undetermined and no specialization is identified. […]

9.7.4 Partial ordering of function templates [temp.func.order]

For the rules in Overload ordering, insert after [temp.func.order]/3:

3 To produce the transformed template, for each type, constant, type template, variable template, or concept template parameter (including template parameter packs ([temp.variadic]) thereof) synthesize a unique type, value, class template, variable template, or concept, respectively, and substitute it for each occurrence of that parameter in the function type of the template. The type replacing a placeholder in the type of the value synthesized for a constant template parameter is a unique synthesized type.

* The transformation preserves each homogeneous function parameter pack and applies the usual transformation to all template parameters, including its count parameter. The parameter pattern is not expanded to the count deduced for a candidate specialization.

9.7.5 Deducing template arguments during partial ordering [temp.deduct.partial]

In [temp.deduct.partial]/8, make the inline change and insert immediately after the sentence shown below:

8 […] Each comparison deduces template arguments for subsequent positions in the template parameter packs (if any) expanded by the function parameter pack. For partial ordering, the pattern of a homogeneous function parameter pack is its parameter pattern ([dcl.fct]). Deduction against this pattern combines deductions for template parameters under the usual consistency requirements. Comparison of two pack patterns does not determine their count arguments from the arity of a candidate specialization. A parameter transformed from a homogeneous function parameter pack remains a parameter transformed from a function parameter pack for the purposes of this subclause. […]

Existing tie-breaks apply without a new preference for homogeneous packs, pending Poll 8 (Proposed EWG polls).

[Drafting note: Whether the invented count is used in types compared for partial ordering under [temp.deduct.partial]/12, including whole function types outside calls, remains unspecified.]

9.8 Declarations

[Drafting note: Wording for the declaration correspondence described in Address-taking remains to be supplied.]

9.9 Feature-test macro [cpp.predefined]

Add a homogeneous function parameter pack feature-test macro to Table 22, Feature-test macros. Options include `__cpp_homo_fn_packs` and `__cpp_homogeneous_function_packs`. SG10 will choose the spelling. Its value remains undetermined.

[Drafting note: Settle the macro spelling with SG10 and supply its value.]

9.10 Annex C (informative) Compatibility [diff.cpp26]

Add a new entry to [diff.cpp26.dcl], before the [dcl.init] entry:

> **Affected subclause:** [dcl.fct]
>
> **Change:** An ellipsis denoting C varargs following a parameter-declaration-list requires a preceding comma. An unnamed parameter declaration ending in an ellipsis can declare a homogeneous function parameter pack.
>
> **Rationale:** Permit unnamed homogeneous function parameter packs.
>
> **Effect on original feature:** Valid C++ 2026 code using comma-less C-varargs declarations may become ill-formed, change meaning, or become incompatible with existing C-varargs definitions. Inserting the comma preserves the original meaning.

10 Conclusion

Further design and wording work depends on EWG direction. We ask EWG to confirm mandatory C-varargs commas and reclamation for unnamed packs and, on that basis, support homogeneous function parameter packs for C++29.

11 Acknowledgments

Thanks to Richard Smith, David Vandevoorde, John Spicer, and Matthias Wippich for impromptu feedback and to Zhihao Yuan for identifying an issue with an earlier `make_array` example.

12 References

[ABSL-LAYOUT] The Abseil Authors. Abseil: layout.h (d522178).
https://github.com/abseil/abseil-cpp/blob/d522178b54087d2c3855484c1899ce99ad278169/absl/container/internal/layout.h

[CASADI-CONCAT] The CasADi developers. CasADi: concatenation overloads (d30fbc4).
https://github.com/casadi/casadi/blob/d30fbc46a6cc7ca18b6e63f63d586069beaacd11/casadi/core/sparsity_interface.hpp#L493-L605

[CASADI-PROXY] The CasADi developers. CasADi: indexing proxy (d30fbc4).
https://github.com/casadi/casadi/blob/d30fbc46a6cc7ca18b6e63f63d586069beaacd11/casadi/core/submatrix.hpp#L98-L99

[CLANG-20-NOTES] Clang 20.1.0 Release Notes.
https://releases.llvm.org/20.1.0/tools/clang/docs/ReleaseNotes.html

[CLANG-DIAGNOSTICS] Diagnostic flags in Clang.
https://clang.llvm.org/docs/DiagnosticsReference.html

[GCC-CXX-DIALECT] Options Controlling C++ Dialect (Using the GNU Compiler Collection).
https://gcc.gnu.org/onlinedocs/gcc/C_002b_002b-Dialect-Options.html

[LOEW-HOMOGENEOUS-OVERLOADS] Tobias Loew. 7 January 2020. How to Define A Variadic Number of Arguments of the Same Type - Part 4.
https://www.fluentcpp.com/2020/01/07/how-to-define-a-variadic-number-of-arguments-of-the-same-type-part-4/
[MAZIERES-PARAM-PACK] David Mazières. June 2022. C++20 idioms for parameter packs.
https://www.scs.stanford.edu/~dm/blog/param-pack.html#homogeneous-function-parameter-packs
[N3096] 1 April 2023. Programming languages - C.
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
[N5054] N5054: Working Draft, Programming Languages - C++.
https://wg21.link/N5054
[P0281R0] James Touton. 2016-01-23. Remove comma elision in variadic function declarations.
https://wg21.link/p0281r0
[P1061R10] Barry Revzin and Jonathan Wakely. 22 November 2024. Structured Bindings can introduce a Pack.
https://wg21.link/p1061r10
[P1141R2] Ville Voutilainen, Thomas Köppe, Andrew Sutton, Herb Sutter, Gabriel Dos Reis, Bjarne Stroustrup, Jason Merrill, Hubert Tong, Eric Niebler, Casey Carter, Tom Honermann, Erich Keane, Walter E. Brown, Michael Spertus, and Richard Smith. 9 November 2018. Yet another approach for constrained declarations.
https://wg21.link/p1141r2
[P1161R3] Corentin Jabot. 22 February 2019. Deprecate uses of the comma operator in subscripting expressions.
https://wg21.link/p1161r3
[P1219-CLANG-TESTS] Historical Clang tests for homogeneous variadic function parameters.
https://github.com/Bekenn/clang/blob/ff4799a4b5b8d071eba265249994aee7b7970258/test/SemaTemplate/cxx2a-func-parm-packs.cpp
[P1219-EWG-RECORD] P1219 Homogeneous variadic function parameters.
https://github.com/cplusplus/papers/issues/297
[P1219R0] James Touton. 8 October 2018. Homogeneous variadic function parameters.
https://wg21.link/p1219r0
[P1219R2] James Touton. 5 October 2019. Homogeneous variadic function parameters.
https://wg21.link/p1219r2
[P2128R6] Mark Hoemmen, Daisy Hollman, Corentin Jabot, Isabella Muerte, and Christian Trott. 14 September 2021. Multidimensional subscript operator.
https://wg21.link/p2128r6
[P2347R2] Corentin Jabot and Bruno Manganelli. 13 October 2021. Argument type deduction for non-trailing parameter packs.
https://wg21.link/p2347r2
[P2662R3] Corentin Jabot and Pablo Halpern. 8 November 2023. Pack Indexing.
https://wg21.link/p2662r3
[P3176R1] Jan Schultke. 22 November 2024. The Oxford variadic comma.
https://wg21.link/p3176r1
[P3776R1] Jan Schultke and Murat Can Çağrı. 9 September 2025. More trailing commas.
https://wg21.link/p3776r1
[REDDIT-ENUM-PACK] 23 January 2023. How can I create a variadic function that involves a parameter pack and takes only 1 type of argument?
https://www.reddit.com/r/cpp_questions/comments/10j2xrh/
[SD10] 2 December 2024. SD-10: Language Evolution (EWG) Principles.
https://wg21.link/sd10
[SO-10] 20 January 2026. Correct syntax for function template with an even number of int arguments?
https://stackoverflow.com/questions/79872247/
[SO-STRING-VIEW-CONCAT] Edison von Myosotis. 29 May 2024. Fold-expression to concatenate string_views.
https://stackoverflow.com/questions/78550299/fold-expression-to-concatenate-string-views
[SO-TYPED-ACCUMULATOR] 16 September 2025. Variadic template function doesn't infer the correct type of the template argument.
https://stackoverflow.com/questions/79766674/